

EX.NO:11 DEVELOP A CRUD APPLICATION IN JAVA USING JAVAFX AND JDBC

Create Database and Table:

```
CREATE DATABASE studentdb;
USE studentdb;
CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  age INT NOT NULL,
  course VARCHAR(100) NOT NULL
);
```

Program:

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.stage.Stage;

import java.sql.*;

public class StudentManagementApp extends Application {

    private Connection connection;

    private TextField idField = new TextField();
    private TextField nameField = new TextField();
    private TextField ageField = new TextField();
    private TextField courseField = new TextField();
    private TextArea displayArea = new TextArea();
```

```

public static void main(String[] args) {
    launch(args);
}

public void start(Stage primaryStage) {
    connectToDatabase();

    // Set up the layout
    GridPane gridPane = new GridPane();
    gridPane.setPadding(new Insets(10));
    gridPane.setHgap(10);
    gridPane.setVgap(10);

    // Set up font
    Font font = new Font("Arial", 14);

    idField.setPromptText("ID (for Update/Delete)");
    idField.setFont(font);
    nameField.setPromptText("Name");
    nameField.setFont(font);
    ageField.setPromptText("Age");
    ageField.setFont(font);
    courseField.setPromptText("Course");
    courseField.setFont(font);

    Button createButton = new Button("Create");
    createButton.setFont(font);
    createButton.setStyle("-fx-background-color: #4CAF50;
-fx-text-fill: white;");
    createButton.setOnAction(e -> createStudent());

    Button readButton = new Button("Display");
    readButton.setFont(font);
    readButton.setStyle("-fx-background-color: #2196F3;

```

```

-fx-text-fill: white;");
    readButton.setOnAction(e -> readStudents());

    Button updateButton = new Button("Update");
    updateButton.setFont(font);
    updateButton.setStyle("-fx-background-color: #FF9800;
-fx-text-fill: white;");
    updateButton.setOnAction(e -> updateStudent());

    Button deleteButton = new Button("Delete");
    deleteButton.setFont(font);
    deleteButton.setStyle("-fx-background-color: #F44336;
-fx-text-fill: white;");
    deleteButton.setOnAction(e -> deleteStudent());

    displayArea.setFont(font);
    displayArea.setEditable(false);
    displayArea.setWrapText(true);

    // Arrange elements in the grid
    gridPane.add(new Label("ID:"), 0, 0);
    gridPane.add(idField, 1, 0);
    gridPane.add(new Label("Name:"), 0, 1);
    gridPane.add(nameField, 1, 1);
    gridPane.add(new Label("Age:"), 0, 2);
    gridPane.add(ageField, 1, 2);
    gridPane.add(new Label("Course:"), 0, 3);
    gridPane.add(courseField, 1, 3);

    // Arrange buttons in two rows with gaps
    gridPane.add(createButton, 0, 4);
    gridPane.add(readButton, 1, 4);
    gridPane.add(updateButton, 0, 5);
    gridPane.add(deleteButton, 1, 5);

```

```

GridPane.setMargin(createButton, new Insets(5, 5, 5, 5));
GridPane.setMargin(readButton, new Insets(5, 5, 5, 5));
GridPane.setMargin(updateButton, new Insets(5, 5, 5, 5));
GridPane.setMargin(deleteButton, new Insets(5, 5, 5, 5));

// Add display area
gridPane.add(displayArea, 0, 6, 2, 1);

// Set up the scene
Scene scene = new Scene(gridPane, 400, 500);
primaryStage.setTitle("Student Management");
primaryStage.setScene(scene);
primaryStage.show();
}

public void connectToDatabase() {
    try {
        // Load the MySQL JDBC driver
        Class.forName("com.mysql.cj.jdbc.Driver");

        // Establish a connection to the database
        connection =
DriverManager.getConnection("jdbc:mysql://localhost:3306
/studentdb", "root", "1234");

        System.out.println("Database connection successful");
    } catch (ClassNotFoundException e) {
        System.err.println("MySQL JDBC Driver not found");
        e.printStackTrace();
    } catch (SQLException e) {
        System.err.println("Database connection failed");
        e.printStackTrace();
    }
}
}

```

```

private void createStudent() {
    String name = nameField.getText();
    int age = Integer.parseInt(ageField.getText());
    String course = courseField.getText();

    String sql = "INSERT INTO students (name, age, course) VALUES
    (?, ?, ?)";

    try (PreparedStatement pstmt =
connection.prepareStatement(sql)) {
        pstmt.setString(1, name);
        pstmt.setInt(2, age);
        pstmt.setString(3, course);
        pstmt.executeUpdate();
        displayArea.setText("Student created successfully.");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

private void readStudents() {
    String sql = "SELECT * FROM students";

    try (Statement stmt = connection.createStatement();
ResultSet rs = stmt.executeQuery(sql)) {
        StringBuilder sb = new StringBuilder();
        while (rs.next()) {
            sb.append("ID: ").append(rs.getInt("id"))
            .append(", Name: ").append(rs.getString("name"))
            .append(", Age: ").append(rs.getInt("age"))
            .append(", Course: ").append(rs.getString("course"))
            .append("\n");
        }
        displayArea.setText(sb.toString());
    } catch (SQLException e) {

```

```

        e.printStackTrace();
    }
}

private void updateStudent() {
    int id = Integer.parseInt(idField.getText());
    String name = nameField.getText();
    int age = Integer.parseInt(ageField.getText());
    String course = courseField.getText();
    String sql = "UPDATE students SET name = ?, age = ?, course = ?
WHERE id = ?";

    try (PreparedStatement pstmt =
connection.prepareStatement(sql)) {
        pstmt.setString(1, name);
        pstmt.setInt(2, age);
        pstmt.setString(3, course);
        pstmt.setInt(4, id);
        pstmt.executeUpdate();
        displayArea.setText("Student updated successfully.");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

private void deleteStudent() {
    int id = Integer.parseInt(idField.getText());
    String sql = "DELETE FROM students WHERE id = ?";

    try (PreparedStatement pstmt =
connection.prepareStatement(sql)) {
        pstmt.setInt(1, id);
        pstmt.executeUpdate();
        displayArea.setText("Student deleted successfully.");
    } catch (SQLException e) {

```

```
        e.printStackTrace();
    }
}
```

```
@Override
public void stop() throws Exception {
    if (connection != null && !connection.isClosed()) {
        connection.close();
    }
    super.stop();
}
}
```